



12

9

1800 sp

نظري

كلية الهندسة المعلوماتية

السنة الرابعة

الأنظمة الموزعة: التنافس،
التكرار، وأنماط تحسين الأداء

د . عمار جوخدار



RB Informatics; 25/5/2026

برمجة تفرعية

محتوى مجاني غير مخصص للبيع التجاري

- في المحاضرات السابقة تعرّفنا على مفهوم التنافس (Concurrency) والمشكلات التي قد تظهر عندما تعمل عدة خيوط تنفيذ أو عمليات بشكل متزامن على مورد مشترك. كما درسنا نموذج المنتج والمستهلك (Producer / Consumer Pattern) باعتباره أحد أشهر أنماط التصميم المستخدمة لتنظيم تبادل البيانات بين المكونات المتوازية ومنع حدوث التضارب أثناء التنفيذ.
- لكن في الأنظمة المؤسسية الحقيقية لا تقتصر مشكلة التنافس على الخيوط والعمليات الموجودة داخل تطبيق واحد أو داخل ذاكرة جهاز واحد، بل تمتد أحياناً إلى مستوى أكبر، حيث تُوزّع البيانات بين عدة قواعد بيانات أو عدة خوادم تعمل معاً ضمن النظام نفسه. وعندها يصبح التحدي ليس فقط تنظيم الوصول إلى البيانات، وإنما أيضاً ضمان بقاء هذه البيانات متوافقة ومتزامنة بين جميع النسخ الموجودة في النظام.
- لذلك سنتناول في هذه المحاضرة مجموعة من الحالات المتقدمة المتعلقة بإدارة التنافس في الأنظمة الموزعة وقواعد البيانات المتعددة، والآليات المستخدمة للحفاظ على صحة البيانات واستمرارية عمل النظام مع تحقيق الأداء والاعتمادية المطلوبة.

التنافس في حال وجود أكثر من قاعدة بيانات

- في الأمثلة السابقة كان المورد المشترك موجوداً ضمن تطبيق واحد أو ضمن قاعدة بيانات واحدة، أما في الأنظمة الكبيرة فقد توجد البيانات نفسها في أكثر من قاعدة بيانات أو أكثر من خادم في الوقت نفسه. ويتم اللجوء إلى هذا الأسلوب لأسباب متعددة، منها زيادة الأمان، وتحسين الأداء، وتسريع عمليات القراءة، وتحقيق الاعتمادية واستمرارية العمل في حال تعطل أحد الخوادم.
- لكن وجود البيانات في أكثر من مكان يضيف نوعاً جديداً من مشكلات التنافس، لأن أي تعديل يتم على إحدى قواعد البيانات يجب أن يصل إلى النسخ الأخرى بطريقة صحيحة ومنظمة. فإذا تم تحديث نسخة من البيانات بينما بقيت النسخ الأخرى تحتوي على القيم القديمة، فقد يحصل المستخدمون على نتائج مختلفة تبعاً لقاعدة البيانات التي يتصلون بها.
- فعلى سبيل المثال : لا تعتمد الأنظمة الضخمة مثل شبكات التواصل الاجتماعي على قاعدة بيانات واحدة فقط، وإنما على عدد كبير من قواعد البيانات الموزعة والمكررة. لذلك قد تتم عملية القراءة من قاعدة بيانات معينة،

بينما تتم الكتابة في قاعدة أخرى، الأمر الذي يفرض الحاجة إلى آليات تضمن انتقال التعديلات ومزامنتها بين هذه القواعد.

- ومن الأمثلة التقليدية على هذه المشكلة الأنظمة المصرفية التي كانت تعتمد سابقاً على وجود قواعد بيانات مستقلة في الفروع المختلفة لأسباب أمنية وتشغيلية. ففي حال انقطاع الاتصال بين أحد الفروع والمركز الرئيسي (Offline)، قد تستمر العمليات المحلية داخل ذلك الفرع، بينما لا تتمكن بقية الفروع من معرفة التعديلات الجديدة مباشرة.
- ولنفترض أن أحد العملاء يملك حساباً مصرفياً في فرع معين، ثم انقطع اتصال هذا الفرع بالشبكة. إذا قام العميل خلال هذه الفترة بسحب مبلغ من المال من فرع آخر، فإن الفرع المنقطع لن يعلم بهذه العملية حتى يعود الاتصال وتتم المزامنة (Synchronization) بين قواعد البيانات. وقد يؤدي ذلك إلى ظهور تعارضات أو أخطاء في الرصيد إذا لم يكن النظام مصمماً لمعالجة هذه الحالات.
- **لهذا السبب** تعتمد المؤسسات حلاً وظيفية (Business Solutions) تختلف من نظام لآخر بحسب طبيعة العمل. فبعض الأنظمة تسمح بتنفيذ العمليات فقط من الفرع المرتبط بالحساب أثناء فترات الانقطاع، وبعضها يفرض قيوداً إضافية مثل تحديد عدد العمليات المسموح بها خلال فترة معينة، ثم إجراء المزامنة بين قواعد البيانات في أوقات محددة.

التكرار (Replication) والنسخ الاحتياطي (Backup) :

من المهم التمييز بين مفهوم التكرار (Replication) والنسخ الاحتياطي (Backup).
فالنسخ الاحتياطي : يهدف إلى حفظ نسخة من البيانات لاستعادتها عند حدوث الأعطال، وغالباً لا تستخدم هذه النسخة في التشغيل اليومي للنظام.
التكرار (Replication) : فيعني وجود البيانات نفسها في أكثر من قاعدة بيانات بهدف المشاركة الفعلية في العمل وتحسين الأداء أو الاعتمادية.

■ وتوجد عدة أساليب للتكرار، منها:

- وجود قاعدة بيانات رئيسية فعالة وأخرى احتياطية غير مستخدمة إلا عند حدوث عطل. في هذه الحالة لا يتم الاستفادة من النسخة المكررة في العمليات اليومية.
- وجود قاعدة مخصصة للكتابة وقاعدة أخرى مخصصة للقراءة، مما يساعد على توزيع الحمل وتحسين الأداء، إلا أن عملية المزامنة بينهما تجعل عمليات الكتابة أكثر تكلفة.
- وجود عدة قواعد بيانات تسمح جميعها بالقراءة والكتابة معاً، وهنا تصبح إدارة التزامن أكثر تعقيداً، لأن أي تعديل يجب أن ينتقل إلى جميع النسخ الأخرى بطريقة تضمن عدم حدوث تعارض أو فقدان للبيانات.

وفي الأنظمة الضخمة قد يقبل النظام أحياناً وجود فروقات مؤقتة بين النسخ المختلفة من البيانات. فمثلاً قد يشاهد المستخدم عدداً مختلفاً قليلاً من التعليقات أو الإعجابات عند استخدام جهازين مختلفين في اللحظة نفسها، لأن المزامنة

لم تكتمل بعد بين جميع النسخ. ويُعد هذا السلوك مقبولاً في بعض الأنظمة الاجتماعية، بينما يكون غير مقبول إطلاقاً في الأنظمة المالية أو المصرفية التي تتطلب درجة أعلى من الدقة والتوافق.

وبشكل عام لا يوجد حل واحد يناسب جميع الأنظمة، وإنما تختلف آليات المعالجة تبعاً لطبيعة العمل ومتطلبات النظام. إلا أن الهدف الأساسي يبقى واحداً: الاستفادة من التوازي والتوزيع دون السماح بأن يؤدي التنافس إلى أخطاء أو تناقضات في البيانات.

التنافس على الموارد وإدارة أولويات التنفيذ

لا تقتصر مشكلات التوازي على التنافس على البيانات وقواعد البيانات فقط، بل قد تظهر أيضاً مشكلة أخرى تتمثل في التنافس على موارد النظام نفسها، مثل المعالج (CPU)، والذاكرة، وعرض الحزمة الشبكية (Bandwidth)، وأجهزة الإدخال والإخراج المختلفة.

فعندما تعمل عدة خيوط تنفيذ (Threads) أو عمليات في الوقت نفسه، تصبح جميعها بحاجة إلى استخدام موارد محدودة. لذلك يجب أن يمتلك نظام التشغيل آليات تضمن توزيع هذه الموارد بصورة عادلة وفعالة، بحيث لا يتمكن أحد الخيوط من احتكار الموارد على حساب بقية الخيوط.

فكرة امتحانية : كيف يحدد نظام التشغيل أولوية تنفيذ الـ Threads ؟

عندما يوجد عدد من الـ Threads الجاهزة للتنفيذ في الوقت نفسه، فإن نظام التشغيل لا يختار أحدها بشكل عشوائي، وإنما يعتمد على مجموعة من المعايير لتحديد أولوية التنفيذ.

1. معيار مدة الانتظار (Waiting Time)

أحد أبسط المعايير هو مدة الانتظار. فكلما ازداد الوقت الذي قضاه الـ Thread في حالة الانتظار، ازدادت فرصته في الحصول على المعالج. ويهدف هذا الأسلوب إلى تحقيق العدالة ومنع بعض الخيوط من البقاء في حالة انتظار لفترات طويلة.

2. معيار استهلاك الموارد

في كثير من الأحيان يعطي نظام التشغيل أولوية أعلى للخيوط التي تحتاج إلى زمن معالجة قصير أو تستهلك موارد أقل. ومن الأمثلة على ذلك الخيوط التي تعتمد بشكل أساسي على عمليات الإدخال والإخراج (I/O)، مثل:

- القراءة والكتابة على القرص الصلب.
- إرسال واستقبال البيانات عبر الشبكة.
- الطباعة على الشاشة.
- التعامل مع أجهزة الإدخال والإخراج المختلفة.

وتحصل هذه الخيوط على أولوية أعلى لأنها لا تبقى لفترات طويلة داخل المعالج، وإنما تنفذ جزءاً صغيراً من العمل ثم تنتقل إلى انتظار جهاز خارجي، مما يسمح بتحقيق استغلال أفضل للـ CPU وزيادة عدد المهام المنجزة خلال الزمن نفسه. لكن الاعتماد على هذا المعيار وحده قد يؤدي إلى مشكلة تسمى الجوع (Starvation)، حيث تبقى الخيوط ذات الاستهلاك المرتفع منتظرة باستمرار بسبب إعطاء الأولوية الدائمة للخيوط الخفيفة.

لذلك تعتمد أنظمة التشغيل عادةً على مزيج من المعايير، بحيث تزداد أولوية الخيوط الثقيلة تدريجياً كلما طال زمن انتظارها، مما يضمن حصولها في النهاية على فرصة للتنفيذ وعدم بقائها منتظرة إلى أجل غير محدود.

3. أولوية المستخدم أو المهمة (User Priority)

قد تمنح بعض الخيوط أولوية أعلى بناءً على طبيعة المهمة التي تنفذها أو أهمية المستخدم الذي أنشأها. **فعلي سبيل المثال**: في تطبيقات الويب والتجارة الإلكترونية توجد خيوط تتعامل مباشرة مع المستخدم وتؤثر على سرعة استجابة النظام، بينما توجد خيوط أخرى تنفذ أعمالاً خلفية لا يلاحظها المستخدم بشكل مباشر. فإذا قام مستخدم بشراء منتج من متجر إلكتروني، فإن الخيوط المسؤولة عن عرض الصفحات والتفاعل مع المستخدم يجب أن تستجيب بسرعة كبيرة، لذلك تُمنح أولوية مرتفعة. أما العمليات اللاحقة مثل حساب تفاصيل الشحن أو إنشاء التقارير أو إرسال الإشعارات، فغالباً تُنفذ في الخلفية (Background) بشكل غير متزامن (Asynchronous)، وبالتالي يمكن منحها أولوية أقل دون التأثير على تجربة المستخدم.

■ مسؤولية نظام التشغيل والمطور

في معظم الحالات يتولى نظام التشغيل إدارة التنافس على موارد المعالج وتنظيم أولويات التنفيذ بين الخيوط المختلفة. أما مسؤولية المهندس أو المصمم فتتمثل في تصميم النظام بطريقة صحيحة، بحيث تُمنح العمليات ذات الأهمية العالية للمستخدم أولوية أكبر، بينما تُنقل العمليات غير العاجلة إلى الخلفية وتنفذ بشكل غير متزامن كلما أمكن ذلك. وبذلك يتحقق التوازن بين سرعة الاستجابة للمستخدمين والاستغلال الفعال لموارد النظام.

تقليل استهلاك الذاكرة باستخدام نمط Lazy List

لا يقتصر تحسين أداء الأنظمة المتوازية على إدارة استهلاك المعالج فقط، بل يشمل أيضاً إدارة استهلاك الذاكرة (Memory) وعرض الحزمة الشبكية (Bandwidth). ففي الأنظمة المؤسسية قد يطلب المستخدم بيانات ضخمة جداً من قاعدة البيانات، وقد يؤدي تحميل جميع هذه البيانات دفعة واحدة إلى استهلاك كميات كبيرة من الموارد دون حاجة فعلية.

فعلي سبيل المثال، قد يطلب المستخدم عرض جميع الأشخاص الذين تزيد أعمارهم عن 15 سنة، وقد يكون عدد النتائج بالملايين. في هذه الحالة لا يكون من المنطقي تحميل جميع السجلات إلى الذاكرة ثم إرسالها إلى المستخدم دفعة واحدة، لأن ذلك سيستهلك مقداراً كبيراً من الذاكرة ويزيد حجم البيانات المنقولة عبر الشبكة.

1. نمط Lazy List :

لمعالجة هذه المشكلة تستخدم العديد من الأنظمة ما يسمى بـ Lazy List، وهي إحدى آليات التصميم التي تعتمد على جلب البيانات تدريجياً بدلاً من تحميلها كاملة منذ البداية. تعتمد الفكرة على تقسيم النتائج إلى دفعات صغيرة (Batches)، بحيث يتم جلب عدد محدود من السجلات في كل مرة، ثم جلب الدفعة التالية عند الحاجة فقط. فعندما يرسل المستخدم استعلاماً كبيراً، يقوم النظام داخلياً بتقسيم النتائج إلى أجزاء دون أن يشعر المستخدم بذلك. فقد يتم جلب أول 20 نتيجة فقط، ثم عند طلب الصفحة التالية يتم جلب 20 نتيجة أخرى، وهكذا حتى انتهاء البيانات.

ولهذا السبب تظهر في كثير من التطبيقات ومواقع الويب أضرار مثل:

Next ■

Previous ■

أو أنظمة التصفح بين الصفحات (Pagination)، والتي تسمح للمستخدم بالانتقال بين أجزاء النتائج بدلاً من تحميلها كاملة دفعة واحدة.

■ الفوائد الناتجة عن Lazy List

يحقق هذا الأسلوب عدة فوائد مهمة:

1. تقليل استهلاك الذاكرة (RAM)

لا يتم تحميل جميع النتائج إلى الذاكرة في الوقت نفسه، وإنما يتم الاحتفاظ بالجزء المطلوب فقط. لذلك يبقى استهلاك الذاكرة محدوداً حتى لو كان حجم النتائج الفعلي كبيراً جداً.

2. تقليل استهلاك الشبكة (Bandwidth)

بدلاً من نقل آلاف أو ملايين السجلات إلى المستخدم، يتم إرسال الجزء المطلوب فقط، مما يقلل كمية البيانات المنقولة عبر الشبكة ويحسن سرعة الاستجابة.

3. توزيع الحمل على المعالج (CPU)

عندما يتم تقسيم النتائج إلى عدة دفعات، فإن معالجة البيانات لا تتم دفعة واحدة، بل تتوزع على عدة عمليات صغيرة متتالية.

فعلي سبيل المثال، إذا كانت النتائج تتكون من 100 سجل ويتم عرض 10 سجلات في كل صفحة، فإن النظام سيعالج هذه البيانات على عشر مراحل مختلفة بدلاً من تنفيذ عملية ضخمة واحدة. وبهذه الطريقة يحصل كل طلب على جزء صغير من وقت المعالج، ثم يفسح المجال لبقية الطلبات قبل تنفيذ الجزء التالي.

■ مثال عملي :

تعد محركات البحث مثلاً واضحاً على هذا الأسلوب. فعند تنفيذ عملية بحث في محرك بحث مثل Google قد تكون النتائج الفعلية بالملايين، لكن المستخدم لا يرى إلا الصفحة الأولى التي تحتوي على عدد محدود من النتائج. ويتم جلب الصفحات التالية فقط عند طلبها.

وتعد الصفحة الأولى من النتائج الأكثر أهمية والأعلى كلفة من ناحية الحسابات والترتيب، لذلك لا يوجد داع لمعالجة جميع النتائج وإرسالها دفعة واحدة طالما أن المستخدم قد لا ينتقل أساساً إلى الصفحات اللاحقة.

■ الهدف من Lazy List

الهدف الأساسي من هذا النمط هو جعل استهلاك الموارد مستقراً وقابلاً للتنبؤ. فبغض النظر عن حجم البيانات الفعلي، يبقى مقدار الذاكرة المستخدمة، وحجم البيانات المنقولة عبر الشبكة، والموارد المطلوبة من المعالج ضمن حدود ثابتة تقريباً لكل طلب.

وبذلك يمكن للنظام خدمة عدد أكبر من المستخدمين بكفاءة أعلى، دون أن يؤدي استعلام ضخم من أحد المستخدمين إلى استهلاك كمية كبيرة من الموارد على حساب بقية المستخدمين.

Keep going

2. نمط Frame: التعامل الذكي مع البيانات

بعد أن رأينا كيف يساهم نمط Lazy List في تقليل استهلاك الموارد من خلال جلب السجلات على دفعات صغيرة، يمكن الانتقال إلى مستوى أكثر دقة من التحسين، وهو التحكم بالحقول (Fields) الموجودة داخل الكائنات نفسها. وهنا يظهر نمط تصميم يسمى Frame .

يقوم هذا النمط على مبدأ بسيط: لا يتم جلب أو حساب أي معلومة إلا عند الحاجة الفعلية إليها (If Needed). فبدلاً من تحميل جميع بيانات الكائن دفعة واحدة، يتم جلب الحقول المطلوبة فقط، بينما تبقى بقية الحقول غير معروفة إلى أن يطلبها البرنامج.

لماذا نستخدم Frame؟

عندما يصل طلب إلى الخادم (Server)، فإن النظام لا يعلم مسبقاً أي الحقول ستحتاجها بقية التوابع أو المكونات البرمجية لاحقاً. لذلك فإن جلب جميع البيانات منذ البداية قد يؤدي إلى تنفيذ استعلامات إضافية غير ضرورية واستهلاك أكبر للذاكرة وقاعدة البيانات.

فعلي سبيل المثال، إذا احتاج أحد التوابع إلى اسم الطالب وكنيته فقط، فلا يوجد داع لجلب علاماته أو عقوباته أو بياناته المالية أو غيرها من المعلومات التي قد لا تُستخدم إطلاقاً أثناء تنفيذ الطلب. ولهذا السبب يعتمد Frame على مبدأ الجلب المتأخر (Lazy Fetch)، حيث يتم جلب المعلومة فقط عندما تصبح مطلوبة فعلاً.

■ الحالات الثلاث لأي حقل

في الكلاسات التقليدية يوجد عادة حالتان فقط للحقل:

■ قيمة موجودة.

■ Null

لكن Frame يعتمد ثلاث حالات مختلفة لكل حقل:

1. قيمة معروفة (Known / Non-Null)

أي أن النظام يعرف قيمة الحقل ويملكها بالفعل.
مثال:

■ اسم الطالب = أحمد.

■ عدد المواد = 6.

2. Null

تعني أن المعلومة معروفة ولكن قيمتها فارغة فعلاً.
مثال:

■ الشخص لا يملك سيارة.

■ الطالب لا يملك رقماً هاتفياً إضافياً.

هنا النظام يعرف الحقيقة وهي أن القيمة غير موجودة.



تعني أن النظام لا يعرف المعلومة أصلاً.
مثال:

- لا نعلم إن كان الشخص يملك سيارة أم لا.
- لم يتم جلب عنوان الطالب بعد.

وهذه الحالة هي أهم إضافة يقدمها Frame لأنها تميز بين "القيمة غير موجودة" و"المعلومة غير معروفة".

■ الحقول الديناميكية

في الكلاسات التقليدية يتم حجز مساحة لجميع الحقول منذ إنشاء الكائن، حتى لو لم تُستخدم. أما في Frame فلا يوجد عدد ثابت من الحقول المحجوزة مسبقاً، بل تُخزن البيانات بشكل ديناميكي يشبه الـ Map أو القواميس المستخدمة في JavaScript. وبالتالي فإن الحقل غير المعروف لا يشغل مساحة في الذاكرة أساساً، مما يؤدي إلى تقليل استهلاك الذاكرة وتحسين الأداء.

■ الجلب الذكي للمعلومات

عند طلب قيمة حقل غير موجودة، يقوم محرك Frame بمحاولة الحصول عليها بأسرع طريقة ممكنة. فإذا أمكن اشتقاق المعلومة من بيانات أخرى متوفرة فلن يتم الرجوع إلى قاعدة البيانات.

على سبيل المثال:

إذا كانت كنية الأب موجودة، بينما كنية الابن غير موجودة، فقد يتمكن النظام من استنتاج الكنية مباشرة دون تنفيذ استعلام جديد على قاعدة البيانات. ولهذا يسعى Frame دائماً إلى الوصول للمعلومة بأقل كلفة ممكنة.

■ العفاريت الثلاثة (Three Demons)

يعتمد Frame Engine على ثلاثة آليات أساسية أطلق عليها في المحاضرة اسم "العفاريت" (Demons)، وهي:

1- If Needed

يُنفذ عندما تُطلب معلومة غير متوفرة.

عندها يحاول النظام:

- حساب القيمة إذا كانت قابلة للحساب.
 - اشتقاقها من بيانات أخرى.
 - جلبها من قاعدة البيانات عند الضرورة.
- أي أن المعلومة لا تجلب إلا عند الحاجة الفعلية إليها.

2- If Added

يُنفذ عند إضافة أو تعديل معلومة.

فعندما تتغير قيمة أساسية داخل الكائن، قد تصبح بعض القيم الأخرى غير صحيحة، لذلك يجب إبطالها وإعادة بنائها إلى حالة Unknown.



3- If Removed

يُنْفَذ عند حذف معلومة أو إزالتها.

ويهدف إلى إزالة أو إبطال جميع البيانات المشتقة التي أصبحت غير صحيحة نتيجة هذا التغيير.

■ الحقول المحسوبة (Computed Fields)

من أهم أفكار Frame أنه لا يفرق بين الحقول (Fields) والتتابع (Methods)، بل يعتبر كليهما مجرد واصفات (Descriptors).

فعلى سبيل المثال، لنفترض وجود مستطيل يحتوي على:

■ Length

■ Width

■ Area

في البرمجة التقليدية غالباً ما يتم تخزين المساحة كقيمة مستقلة.

أما في Frame فإن المساحة تعتبر حقلاً محسوباً:

$$\text{Area} = \text{Length} \times \text{Width}$$

وعندما يطلب البرنامج قيمة المساحة لأول مرة، يقوم If Needed بحسابها، بعد ذلك يمكن تخزينها مؤقتاً للاستفادة منها لاحقاً.

ماذا يحدث عند تعديل الطول أو العرض؟

إذا تغير الطول أو العرض تصبح قيمة المساحة القديمة غير صحيحة. لذلك يقوم If Added أو If Removed بإزالة قيمة المساحة وإعادة إدخالها إلى حالة Unknown.

وعندما تطلب المساحة مرة أخرى يتم حسابها من جديد.

وبذلك لا تتم الحسابات إلا عند الحاجة، ولا يتم الاحتفاظ بقيم قديمة أو غير صحيحة.

أمثلة أخرى على الحقول المحسوبة

● معدل الطالب

بدلاً من إعادة حساب المعدل عند كل تغيير في علامة مادة، يمكن اعتبار المعدل حقلاً محسوباً.

فعندما تتغير إحدى العلامات يتم فقط إبطال قيمة المعدل.

أما إعادة الحساب فتتم عند طلب المعدل فعلياً.

● مساحة المستطيل

إذا تغير الطول أو العرض:

■ تُزال قيمة المساحة.

■ تبقى في حالة Unknown.

■ يعاد حسابها عند الطلب فقط.

● أسعار العملات

بعض القيم ثابتة نسبياً مثل سعر الافتتاح اليومي، بينما توجد قيم أخرى تتغير باستمرار وتتطلب جلباً متكرراً من

مصادر خارجية. ويستطيع Frame التعامل مع كل حالة بالطريقة الأنسب.

مثال على إدارة التغييرات

لنفترض أن بيانات أحد الطلاب تشير إلى أنه يعيش في سوريا، وتتضمن:

- المحافظة.
- الحي.
- الشارع.
- رقم الهاتف المحلي.

إذا تغيرت دولة الإقامة إلى الولايات المتحدة مثلاً، فإن كثيراً من هذه المعلومات لم تعد صالحة.

بدلاً من محاولة تحديث جميع الحقول يدوياً، يقوم النظام بإزالة الحقول المرتبطة بالعنوان القديم وإعادتها إلى حالة Unknown.

وبذلك يتجنب الاحتفاظ بمعلومات قديمة أو خاطئة إلى أن يتم إدخال البيانات الجديدة.

مزاي Frame

يوفر هذا النمط مجموعة من الفوائد المهمة:

- تحسين الأداء من خلال Lazy Fetch.
- تقليل عدد الاستعلامات المرسلة إلى قاعدة البيانات.
- تخفيض استهلاك الذاكرة.
- توحيد التعامل مع الحقول والتوابع ضمن نموذج واحد.
- زيادة مرونة التعامل مع البيانات الديناميكية.
- تقليل حجم الكود لأن جزءاً كبيراً من المنطق موجود مسبقاً داخل Frame Engine.
- زيادة موثوقية النظام نتيجة تقليل حجم الكود المطلوب من المبرمج.

ولهذا السبب يُنظر إلى Frame على أنه نموذج أكثر ذكاءً من الكلاسات التقليدية، لأنه يتعامل مع البيانات بطريقة قريبة من طريقة تفكير الإنسان؛ فلا يبحث عن المعلومة، ولا يحسبها، ولا يحدثها إلا عندما تصبح هناك حاجة فعلية لذلك.

■ موثوقية البرمجيات (Reliability) واتجاهات التطور الحديثة

لا تهدف أنماط التصميم الحديثة مثل Frame إلى تحسين الأداء فقط، وإنما تسهم أيضاً في زيادة موثوقية البرمجيات (Reliability). فكلما كان النظام أبسط وأكثر تنظيماً، أصبحت احتمالية احتوائه على أخطاء أقل، وأصبح فهمه وصيانته أسهل.

العلاقة بين حجم الكود والموثوقية :

من المبادئ المهمة في هندسة البرمجيات أن موثوقية النظام ترتبط بشكل مباشر بحجم الكود البرمجي. فكلما ازداد حجم الكود:

- ازدادت احتمالية وجود أخطاء برمجية.
- ازدادت صعوبة المراجعة والاختبار.
- أصبحت عملية الصيانة أكثر تعقيداً.

أما عندما يكون الكود صغيراً وواضحاً وقابلاً للفهم من قبل المطور، فإن احتمالية اكتشاف الأخطاء وتصحيحها تصبح أكبر، وبالتالي ترتفع موثوقية النظام. ولهذا السبب تسعى الاتجاهات الحديثة في تطوير البرمجيات إلى تقليل كمية الكود المكتوب يدوياً والاعتماد على أطر العمل (Frameworks) والمحرركات الجاهزة التي تتولى جزءاً كبيراً من المهام بشكل تلقائي.

كيف يمكن زيادة موثوقية البرامج؟

هناك عدة أساليب تساعد على زيادة موثوقية البرمجيات، منها:

1- تقليل حجم الكود

كلما كان الكود أصغر كان أسهل في القراءة والمراجعة والاختبار. ولهذا السبب يفضل استخدام:

- أطر العمل الجاهزة (Frameworks).
- أنماط التصميم المتقدمة مثل Frame.
- الأدوات التي تولد أجزاء من الكود تلقائياً.

2- استخدام اللغات التفسيرية (Interpreted Languages)

بعض اللغات الحديثة مثل JavaScript وLisp تسمح ببناء نماذج ديناميكية للبيانات وتوفر مرونة أكبر مقارنة باللغات التقليدية.

فعلى سبيل المثال، يمكن في JavaScript التحقق من وجود الحقل فعلياً باستخدام مفاهيم مثل:

- Haskey
- property existence

بدلاً من حجز مساحة مسبقة لجميع الحقول كما يحدث في كثير من اللغات الكلاسيكية.

3- استخدام اللغات التصريحية (Declarative Languages)

في اللغات التصريحية يصف المبرمج النتيجة المطلوبة بدلاً من وصف خطوات التنفيذ التفصيلية. ومن الأمثلة عليها:

- HTML
- Prolog

وهذا يؤدي غالباً إلى تقليل حجم الكود وزيادة وضوحه.

كن صانع الفوز في مسيرتك،

ولا تقف في صفوف المتفرجين.



4- الاعتماد على Low-Code Frameworks

تسمح بعض الأطر الحديثة بتوليد أجزاء كبيرة من التطبيق تلقائياً، مثل:

- واجهات المستخدم.
- قواعد البيانات.
- المخططات (Schemas).
- نماذج الإدخال.

مما يقلل كمية الكود المكتوب يدوياً ويرفع مستوى الاعتمادية.

5- استخدام Frame بدلاً من الكلاسات التقليدية عندما تكون البيانات ديناميكية ومعقدة.

مقارنة بين اللغات التقليدية واللغات الحديثة

يرى هذا التوجه أن بعض اللغات الحديثة أكثر ملاءمة للتعامل مع البيانات الديناميكية من اللغات التقليدية. فعلى سبيل المثال:

- في JavaScript يمكن أن يكون الحقل غير موجود أساساً داخل الكائن.
 - أما في اللغات الكلاسيكية مثل Java غالباً يتم تعريف الحقول مسبقاً حتى لو كانت قيمتها Null.
- لذلك تتوافق اللغات الديناميكية بشكل أكبر مع فلسفة Frame التي تعتمد على وجود حالة ثالثة للبيانات هي Unknown.

تصنيف البرمجيات وفق طبيعة المدخلات

ذكرنا في المحاضرات السابقة تصنيفاً عاماً للبرمجيات اعتماداً على طبيعة المدخلات التي تتعامل معها، وهي تقسم الى ثلاثة مستويات :

1 | الأنظمة القابلة للعد (Countable) :

وهي الأنظمة التي تمتلك عدداً محدوداً وواضحاً من الحالات الممكنة. في هذه الأنظمة يمكن تمثيل جميع الحالات باستخدام تعليمات شرطية من نوع:

If → Then → Else

مثل:

- نماذج الإدخال البسيطة.
 - الأنظمة ذات القواعد المحددة مسبقاً.
- وبما أن عدد الحالات محدود، يمكن وصفها بشكل مباشر باستخدام الشروط والقواعد التقليدية.

2 | الأنظمة القابلة للعد بشكل عودي (Recursively Enumerable) :

في هذا النوع لا يكون عدد الحالات محدوداً، بل يمكن أن يكون غير منتهٍ، لكن يمكن وصفه بمجموعة من القواعد الرسمية.

ومن أهم الأمثلة:

- المترجمات (Compilers).
 - محللات اللغات البرمجية (Parsers).
- فعلى الرغم من أن عدد البرامج الممكن كتابتها لا نهائي تقريباً، إلا أن جميعها تخضع لقواعد نحوية محددة يمكن تمثيلها بواسطة نماذج رياضية ورسمية.
- ولهذا السبب تعتمد المترجمات على النماذج الشكلية (Formal Models) بدلاً من الاعتماد فقط على سلاسل طويلة من تعليمات If-Then-Else.

3 الأنظمة غير القابلة للعد (Non-Enumerable) :

وهي الأنظمة التي تتعامل مع مدخلات لا يمكن وصف جميع احتمالاتها مسبقاً.

ومن أمثلتها:

- الصور.
- الفيديو.
- الصوت.
- اللغة الطبيعية.

فلا يمكن كتابة جميع الاحتمالات الممكنة لهذه البيانات ضمن قواعد تقليدية ثابتة.

ولهذا السبب ينتمي هذا النوع من المشكلات إلى مجال:

- الذكاء الاصطناعي (Artificial Intelligence).
- تعلم الآلة (Machine Learning).
- الرؤية الحاسوبية (Computer Vision).
- التعرف على الأنماط (Pattern Recognition).

حيث يتم الاعتماد على التعلم والاستنتاج الإحصائي بدلاً من الاعتماد على القواعد البرمجية الصريحة فقط.

خلاصة

تتجه هندسة البرمجيات الحديثة نحو بناء أنظمة أكثر ذكاءً وأعلى موثوقية من خلال تقليل حجم الكود، والاعتماد على الأطر الجاهزة وأنماط التصميم المتقدمة مثل Frame، واستخدام نماذج برمجية مناسبة لطبيعة المشكلة. فبينما تكفي القواعد التقليدية لمعالجة المشكلات القابلة للوصف والعد، تتطلب المشكلات ذات المدخلات المعقدة وغير المحدودة تقنيات أكثر تقدماً تعتمد على الذكاء الاصطناعي والتعلم الآلي.

الفائزون والخاسرون يتشاركون الأهداف نفسها**لكن الفرق في التخطيط والعمل والاجتهاد.**